Olist Data Platform

Full-Stack Data Platform: Lakehouse to Power BI

Project documentation

Databricks | PySpark | Delta Lake | Unity Catalog | Power BI

Repository: github.com/goldenFlori/olist-data-platform

September 2026

Scope: raw-file ingestion, medallion lakehouse, dimensional modeling, incremental processing, data quality, Databricks optimization, and Power BI reporting.

Contents

1. Architecture

The platform follows a medallion architecture on Databricks, using Unity Catalog for governance and Delta Lake for storage.

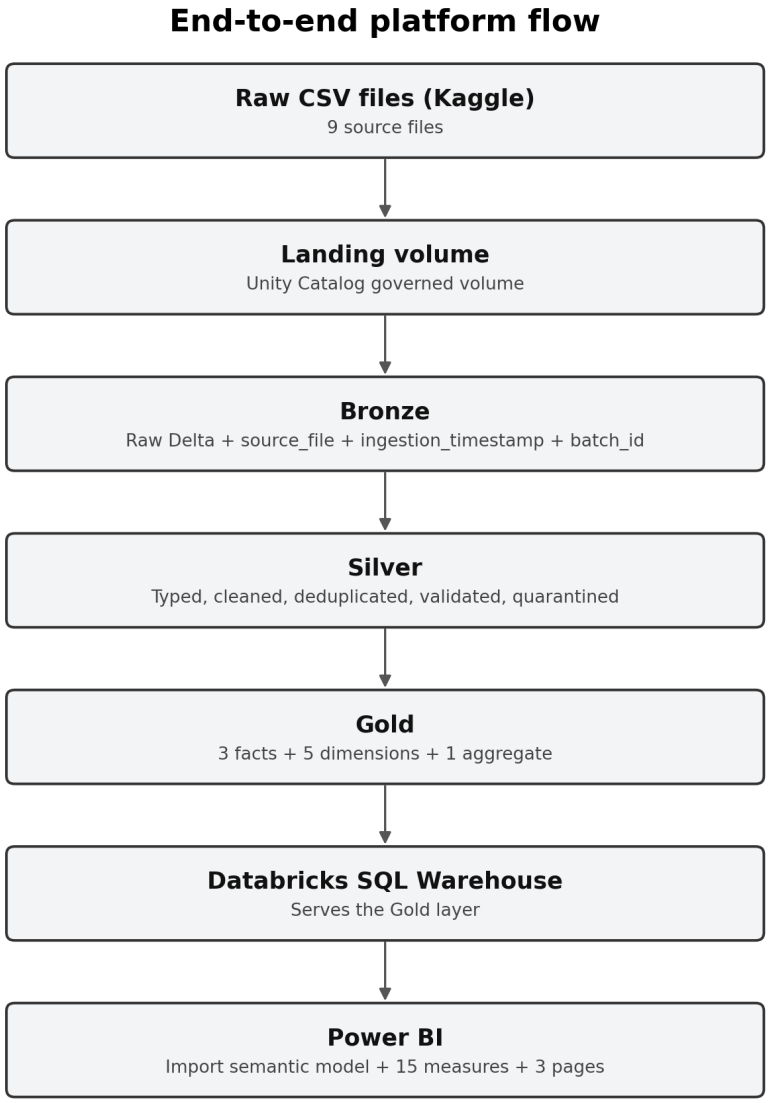

Figure 1. End-to-end lakehouse and reporting flow.

Why layers. Bronze provides the audit trail, Silver centralizes data-quality and business-cleaning rules, and Gold exposes a reporting-ready dimensional model. Keeping these responsibilities separate avoids repeating transformations in Power BI and makes reconciliation back to the source possible.

Storage and idempotency. Tables are stored in Delta Lake. Delta provides ACID transactions, schema-aware writes, MERGE support for incremental upserts, and time travel. Bronze ingestion is reproducible, and incremental fact loading is idempotent so rerunning an increment does not duplicate or double count records.

Governance. Unity Catalog provides the three-level namespace (`catalog.schema.table`) and a governed landing volume. Raw files are stored in the Unity Catalog volume rather than accessed through credentials embedded in notebooks.

Partitioning and optimization. The geolocation source is the largest input at roughly one million rows and is partitioned by state in Bronze. Gold fact tables are later compacted and optimized with Z-ORDER where appropriate.

Orchestration. The pipeline is defined as an ordered job: Bronze -> Silver -> Gold -> Data Quality. Downstream tasks wait for upstream layers to succeed.

2. Data model

The Gold layer uses a dimensional model with three fact tables, five conformed dimensions, and one pre-aggregated reporting table. Each fact keeps its own business grain so measures are evaluated at the correct level.

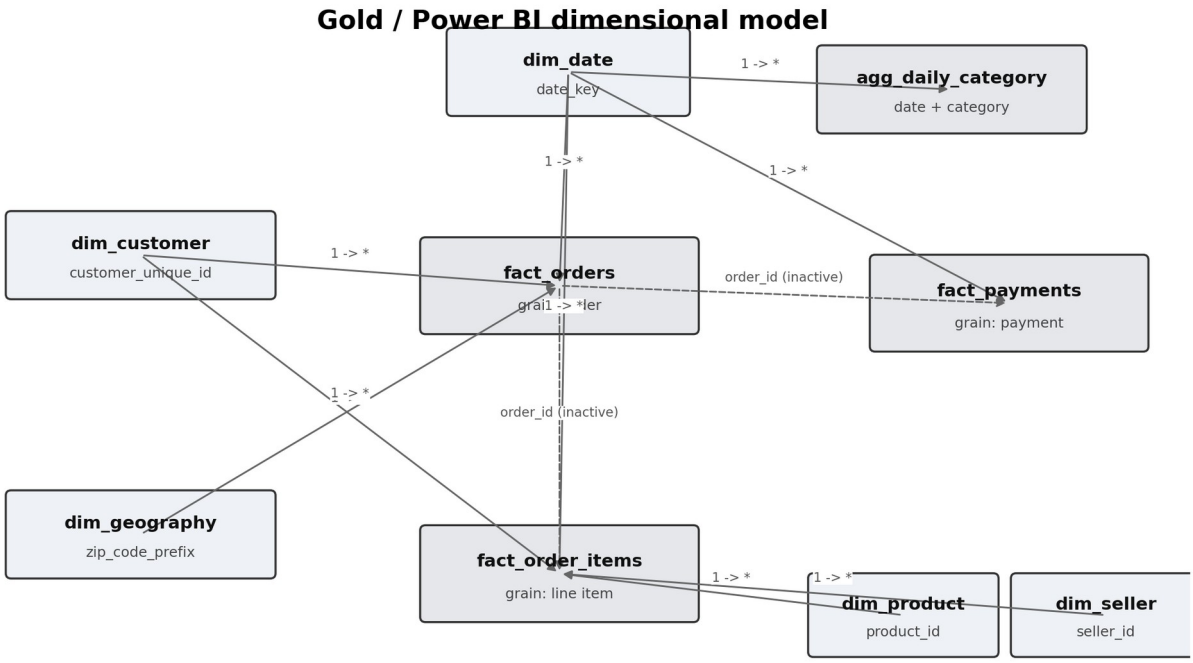

Figure 2. Multi-fact dimensional model with conformed dimensions.

Fact tables

Fact	Grain	Main measures / attributes
`fact_orders`	One row per order	Review score, delivery funnel times, on-time flag, order status
`fact_order_items`	One row per line item	Item price, freight value
`fact_payments`	One row per payment	Payment value, payment type, instalments

Dimensions

Dimension	Key	Notes
`dim_customer`	`customer_unique_id`	Represents the real customer across multiple orders
`dim_product`	`product_id`	English category and product attributes
`dim_seller`	`seller_id`	Seller city and state
`dim_date`	`date_key`	Continuous date dimension covering the full data range; marked as a date table in Power BI
`dim_geography`	`zip_code_prefix`	One resolved coordinate/location row per ZIP prefix

The Gold layer also contains `agg_daily_category`, a pre-aggregated table for daily category reporting.

Why three facts

The exercise requires separate item- and payment-grain facts. I added `fact_orders` because order count, delivery timing, on-time status, and review score are order-level attributes. Putting those measures on the item fact would repeat order-level values once per line item.

Why `customer_unique_id`

In the Olist source, `customer_id` is tied to an order-specific customer record. A returning person can therefore receive a different `customer_id` on a later order. `customer_unique_id` identifies the real customer, so it is the business key of `dim_customer`. It is carried into the order and item facts to support distinct-customer, repeat-customer, and customer-level revenue analysis. The original `customer_id` is retained in the order fact for source traceability.

Fan-out: why items and payments remain separate

An order can contain multiple line items and multiple payment records. Joining the two facts directly by `order_id` creates a many-to-many fan-out: payment rows repeat for each item and item rows repeat for each payment.

The measured impact is material:

- Correct item revenue: about **BRL 13.6M**.
- Item revenue after the fan-out join: about **BRL 14.2M** (**1.05x inflation**).
- Correct payment value: about **BRL 16.0M**.
- Payment value after the fan-out join: about **BRL 20.3M** (**1.27x inflation**).

For that reason, revenue is measured from `fact_order_items` and payment value from `fact_payments`. Each reconciles to the correct total at its own grain. The residual difference between the true item and payment totals is expected because payment value is not the same business measure as item price; it can include freight and other payment effects.

3. Assumptions

Currency is Brazilian Real (BRL). The source does not include an explicit currency field. Monetary values are treated as BRL because the dataset represents domestic Brazilian e-commerce activity.

Products without a category are kept. Products with missing category information remain in the model and are labelled `uncategorized`, rather than being removed and understating sales.

Undelivered orders are kept. Orders without a delivery date remain in the dataset. `order_status` distinguishes delivered, cancelled, unavailable, and in-progress orders, while delivery KPIs only use records for which the relevant timestamps exist.

Incremental cutoff is 2017-12-31. The incremental simulation first loads history up to this cutoff, then processes later records as arriving data. Subsequent incremental runs use the stored watermark rather than restarting from the fixed cutoff.

Delivery funnel metrics are pipeline logic. Approval time, handling time, shipping time, total delivery time, and variance against the estimated delivery date are derived in Silver, not in Power BI, so every consumer uses the same definitions.

4. Data quality issues found

Every source was profiled for missing values, duplicates, and referential-integrity problems before the cleaning rules were applied.

Order reviews contained a multiline CSV parsing problem. An initial row count of 104,162 did not match the documented 99,224 reviews, and values such as dates/text appeared in `review_score`. Some free-text comments contain newlines and quotes. Reading the file with the correct multiline/escape options realigned the columns; after deduplication and quarantine, the clean review table contains 98,410 rows.

Order reviews contain duplicates and orphans. There are repeated `review_id` values, including records with different content. Reviews are deduplicated by keeping the most recent record. Rows with null `order_id`

or null review score are quarantined. Reviews whose `order_id` does not exist in the orders source do not enter `fact_orders`, because the Gold fact is built from orders and joins reviews onto that order population.

A seller city contained a ZIP-like numeric value. The value `04482255` appeared in the city field. The seller's valid ZIP prefix was used to resolve the corresponding city from geolocation, recovering Rio de Janeiro. The rule is generalized so numeric city values can be repaired from ZIP-based geography.

Geolocation had inconsistent city formatting. Many ZIP prefixes mapped to city strings that differed only by accents, apostrophes, case, or spacing. City text is normalized and the most frequent city representation is selected per ZIP prefix. The roughly one-million-row geolocation source resolves to about 19,000 ZIP-level records.

Payments with non-positive value are quarantined. Nine payment rows have a value of zero or less and are treated as invalid financial records. `not_defined` payment types are retained when the payment value itself is valid.

Products with missing descriptive attributes are retained. Missing category/descriptive fields do not cause real products and sales to be dropped.

Referential integrity. The Data Quality gate currently checks **seven fact-to-dimension relationships** and reports zero orphan keys across those implemented checks on the current run.

5. Data quality gate

The final orchestration task runs automated checks and fails the job when a required rule is breached. The gate covers:

1. **Row count per layer:** expected tables must not be empty.
2. **Freshness:** the latest order date must be within the expected source-data range.
3. **Dimension-key uniqueness:** each dimension business key must be unique.
4. **Referential integrity:** seven implemented fact-to-dimension relationships are checked for orphan keys.
5. **Range checks:** item price, freight value, and review score are validated against allowed ranges.

The current run passes these checks.

6. Incremental processing

Incremental processing is driven by `order_purchase_timestamp`.

1. A history run loads records up to the initial cutoff date.
2. The control layer stores the last successfully processed timestamp/date as a watermark.
3. An incremental run reads the current watermark before filtering the next slice.
4. The fact is written with Delta `MERGE` on the business key, so rerunning the same increment updates matching rows instead of inserting duplicates.
5. After a successful write, the watermark advances to the latest processed value.

This separates the one-time historical backfill from subsequent arriving-data loads while preserving idempotency. Incremental executions also record operational metrics such as rows read, rows written, rows quarantined, and duration in the control log.

7. Power BI semantic model

Power BI connects only to Gold through a Databricks SQL Warehouse and uses **Import mode**. Import is appropriate because the Olist dataset is historical and relatively small for VertiPaq; it provides faster interactive report performance without requiring a source round trip for every visual interaction.

The semantic model uses single-direction dimension-to-fact filtering, and `dim_date` is marked as the date table. The Import model is approximately **24 MB**.

The report contains **15 DAX measures**, covering the required areas:

- Revenue, freight cost, and average order value.

- Order count and distinct customer count at the correct grain.
- On-time delivery rate.
- Average review score and the share of scores 1-2.
- Repeat customer rate.
- Month-over-month and year-over-year comparisons.
- Payment totals used for reconciliation against the payment fact.

Seller-state row-level security is implemented so the demo role sees only the relevant seller state.

8. Power BI report

The report contains three pages, each framed around a business question.

Executive Overview - "How is the business performing overall?" Headline KPIs, revenue trend, category contribution, and state breakdown. The trend shows strong scaling through 2017-2018 and a visible Black Friday 2017 spike.

Delivery & Operations - "Are we meeting delivery expectations?" On-time delivery, average delivery time, delivery variance, state-level delivery performance, and worst-performing routes. The on-time rate is about 92%, and actual delivery is roughly 11 days earlier than the estimate on average.

Product & Seller Performance - "Which products and sellers drive value?" Review score, low-review share, repeat-customer rate, top categories, and seller contribution.

The item-revenue and payment-value measures remain sourced from their separate fact tables so the report avoids the fan-out problem described above.

9. Performance

Performance work is implemented at the backend and semantic-model levels.

Backend. A controlled benchmark is run before and after Delta `OPTIMIZE` + Z-ORDER. The measured revenue-by-date query improved from **0.84 seconds to 0.63 seconds**, a **25.3% improvement**. Z-ORDER colocates related column values so the Delta engine can use **data skipping** to avoid reading files that cannot contain relevant values. It is not the same mechanism as partition pruning. Geolocation is physically partitioned by state in Bronze, while Z-ORDER is applied to appropriate Gold fact columns.

Model. Only reporting-relevant Gold columns are loaded into Power BI, reducing unnecessary cardinality. The resulting Import model is approximately **24 MB**.

Report. Import mode keeps report interactions local to the VertiPaq model rather than issuing a source query for every visual interaction.

10. Challenges and lessons learned

Multiline review parsing. The review row-count mismatch was the clue that the CSV had not been parsed correctly. Inspecting the score column exposed shifted values caused by multiline free text. The lesson was to profile the actual source rather than trusting documented row counts.

Customer grain. The first customer dimension used `customer_id`, which produced an order-specific, effectively one-to-one customer dimension and prevented a correct repeat-customer calculation. Moving the dimension to `customer_unique_id` fixed the grain and enabled customer-level analysis.

Multi-fact modeling. Keeping item, payment, and order measures at their natural grains was necessary to avoid fan-out and to make Power BI measures reconcile correctly.

Databricks Free Edition constraints. The five-task concurrency limit led to sequential layer execution rather than parallel tasks. Managed tables were used because external storage locations were not available in the chosen environment.

Power BI on Mac. Power BI Desktop is Windows-only, so the report was developed in a Windows virtual machine connected to the Databricks SQL Warehouse.

Source basis. Prepared against the Full Stack Data Platform exercise requirements, the current repository structure/implementation provided for this project, and the verified project notes supplied during implementation.